

Hybrid Classical + Quantum Optimization for Distributed Order Management

A Quantum-Enhanced Decision Support System for Supply Chain Optimization

Title Page

Hybrid Classical + Quantum Optimization for Distributed Order Management

A Quantum-Enhanced Decision Support System for Supply Chain Optimization

Team Contribution

Shaik Bushra Faizi

- Quantum Optimization
- Dashboard Development
- AI Assistant

Saketh Ram

- Data Processing
- Optimization Pipeline

Pavan Kumar

- Documentation
- Testing

Technology Stack

Python | Streamlit | OR-Tools | Qiskit | Plotly | Pandas

Project Category

Quantum Computing + Artificial Intelligence + Supply Chain Optimization

1. Abstract

Modern supply chains require intelligent decision-making to efficiently manage customer orders across multiple warehouses and distribution centers. Distributed Order Management (DOM) is a complex optimization problem where organizations must decide the best fulfillment location for each order while considering several constraints such as inventory availability, warehouse capacity, transportation cost, delivery requirements, and business priorities.

This project presents a **Hybrid Classical + Quantum Optimization approach** for solving the Distributed Order Management problem.

The system combines:

- Classical optimization using **Google OR-Tools**
- Quantum optimization using **QAOA (Quantum Approximate Optimization Algorithm)** implemented using **Qiskit**

The developed Streamlit dashboard provides an interactive decision-support platform that enables users to analyze optimization results, compare classical and quantum approaches, visualize operational metrics, and understand recommendations through the **Qora AI Assistant**.

The objective of this system is to:

- Maximize order fulfillment
 - Improve fill rate
 - Reduce logistics cost
 - Minimize penalty cost
 - Optimize warehouse utilization
 - Provide explainable supply chain decisions
-

2. Introduction

2.1 Background

Large enterprises such as Nestlé operate complex global supply chains consisting of:

- Manufacturing plants

- Distribution centers
- Warehouses
- Transportation networks
- Customer delivery systems

Every customer order requires a critical decision:

"Which warehouse should fulfill this order?"

Selecting the wrong warehouse may result in:

- Higher transportation cost
- Inventory imbalance
- Delivery delays
- Increased penalty cost
- Reduced customer satisfaction

Therefore, an intelligent optimization system is required to make efficient fulfillment decisions.

2.2 Distributed Order Management (DOM)

Distributed Order Management is a supply chain strategy that determines:

- Where inventory is available
- Which location should fulfill an order
- How orders should be allocated
- How business objectives can be optimized

DOM balances:

Customer Satisfaction
+
Operational Efficiency
+
Cost Reduction

3. Problem Statement

Given:

- Customer orders
- Product demand
- Warehouse inventory

- Warehouse capacity
- Transportation cost
- Delivery constraints

The objective is:

Find the optimal assignment of customer orders to distribution centers while maximizing business value and minimizing operational cost.

4. Objectives

The major objectives of this project are:

Business Objectives

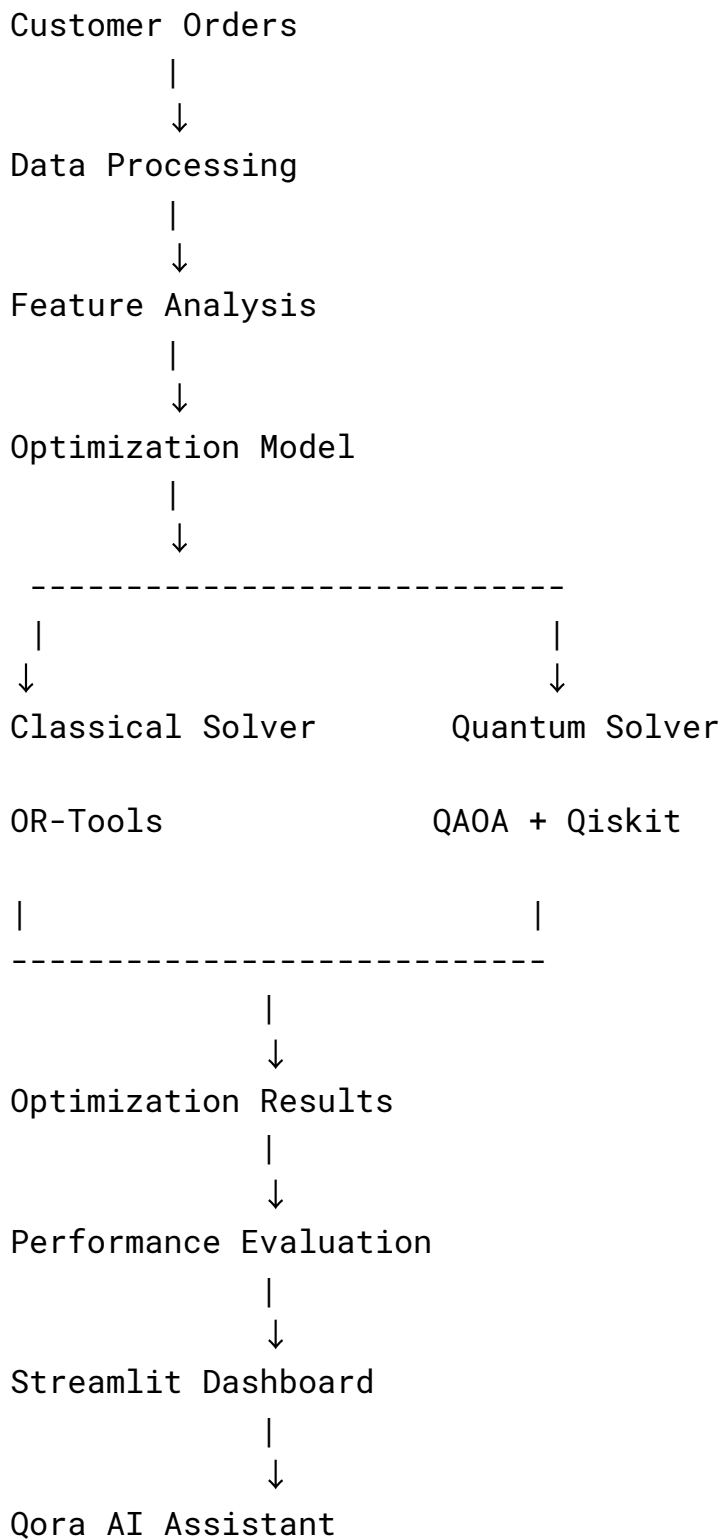
- ✓ Maximize fulfilled customer orders
- ✓ Improve order fill rate
- ✓ Reduce transportation cost
- ✓ Minimize unfulfilled order penalties
- ✓ Balance warehouse workload

Technical Objectives

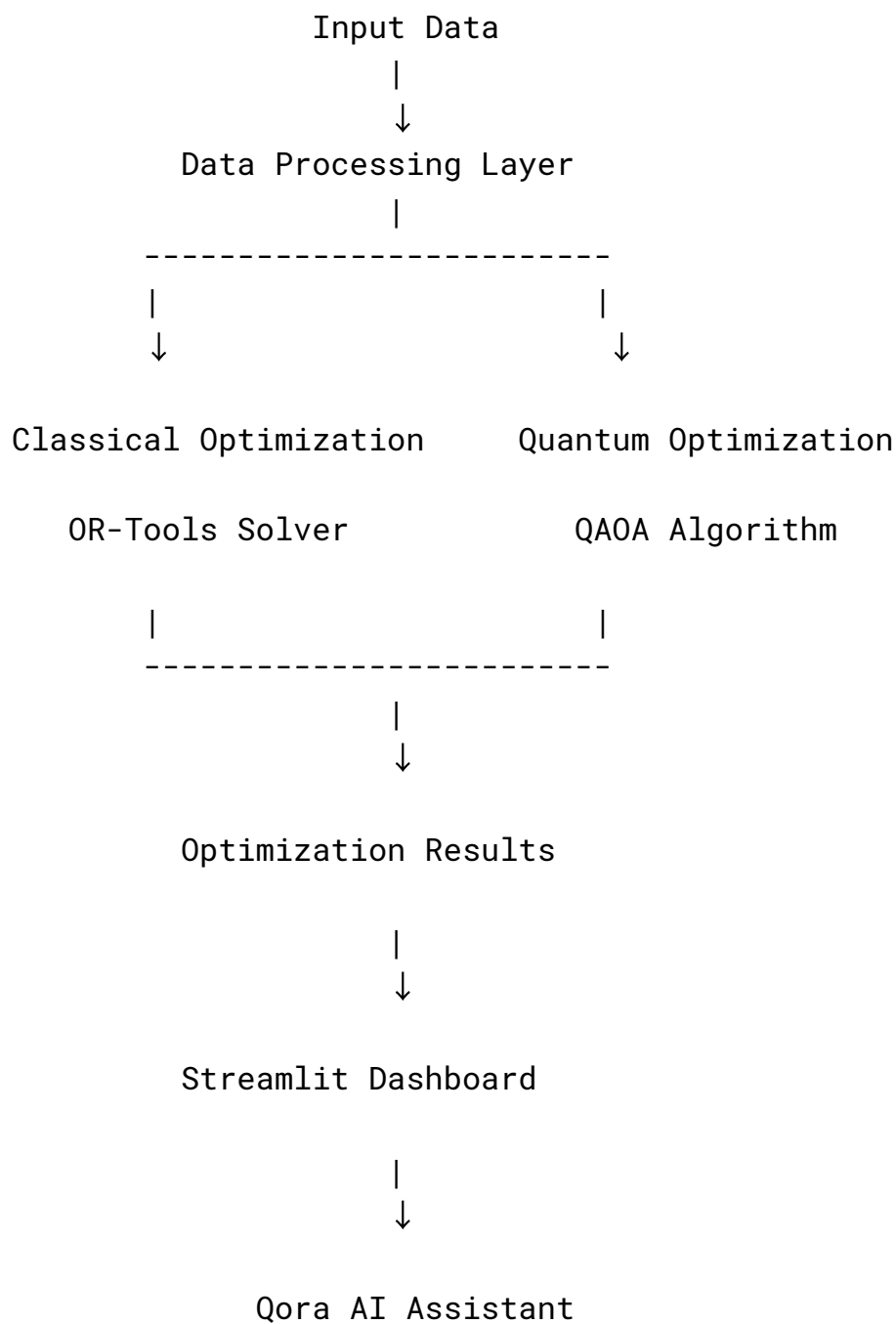
- ✓ Develop classical optimization model
 - ✓ Implement quantum optimization formulation
 - ✓ Compare classical and quantum approaches
 - ✓ Build interactive visualization dashboard
 - ✓ Provide explainable AI assistance
-

5. Project Workflow

The complete workflow of the system:



6. System Architecture



7. Technology Stack

Technology	Purpose
Python	Core programming language
Pandas	Data processing
NumPy	Numerical operations
Streamlit	Web dashboard development
Plotly	Interactive visualization
OR-Tools	Classical optimization solver
Qiskit	Quantum computing framework
QAOA	Quantum optimization algorithm
CSV	Dataset storage

8. Dataset Description

The optimization system uses order and warehouse information.

Dataset Features

Feature	Description
Plant	Distribution center identifier
Material Number	Product SKU
Order ID	Customer order identifier
Quantity	Required product quantity
Inventory	Available warehouse stock
Revenue	Business value of order
Shipping Cost	Transportation expense
Penalty	Cost of unfulfilled order
Delivery Date	Customer delivery requirement

9. Data Processing

The data processing pipeline includes:

Data Cleaning

- Removing missing values
- Correcting data types
- Handling invalid records

Feature Preparation

Preparing:

- Demand information
- Inventory information
- Cost information
- Revenue information

Optimization Input Generation

Converted into mathematical optimization variables.

10. Mathematical Formulation

10.1 Decision Variable

The decision variable represents order assignment.

$$x_{ij} = 1$$

If order i is assigned to warehouse j

$$x_{ij} = 0$$

Otherwise

10.2 Objective Function

The goal is to maximize business value.

Maximize:

Business Value

=

Revenue

-

Shipping Cost

-

Penalty Cost

Mathematically:

[

Maximize $Z =$

$\sum \text{Revenue} -$

$\sum \text{ShippingCost} -$

$\sum \text{Penalty}$

]

10.3 Constraints

Constraint 1: Warehouse Capacity

$\text{Assigned Orders} \leq \text{Warehouse Capacity}$

Each warehouse can process only limited orders.

Constraint 2: Inventory Constraint

$\text{Demand} \leq \text{Available Inventory}$

Available stock should satisfy customer demand.

Constraint 3: Order Assignment

$\text{One Order} \rightarrow \text{One Warehouse}$

Each order should have an optimal fulfillment location.

Constraint 4: Binary Decision

$x_{ij} \in \{0, 1\}$

Assignment is either selected or not selected.

11. Classical Optimization Approach

OR-Tools Solver

Google OR-Tools is used for classical optimization.

Workflow:

Input Dataset



Create Decision Variables



Define Objective Function



Apply Constraints



Run Solver



Generate Optimal Assignment

Advantages:

- Fast
 - Reliable
 - Industry proven
 - Suitable for current-scale problems
-

12. Quantum Optimization Approach

QAOA (Quantum Approximate Optimization Algorithm)

QAOA is a hybrid quantum-classical algorithm designed for optimization problems.

Quantum Workflow

Optimization Problem



Convert to QUBO



Generate Hamiltonian



Create Quantum Circuit



Apply QAOA



Measure Quantum State



Obtain Solution

Why QAOA?

Large optimization problems contain huge search spaces.

Traditional methods may become computationally expensive as problem size increases.

Quantum optimization explores optimization landscapes using quantum principles and may provide advantages for future large-scale problems.

13. Implementation Details

Backend

Python
Pandas
NumPy
OR-Tools
Qiskit

Frontend

Streamlit

Plotly Interactive Dashboard

AI Layer

Qora AI Assistant

Knowledge-based optimization explanation system

14. Dashboard Modules

The application contains six major modules.

Tab 1: Dashboard

Provides:

- Total orders

- Selected orders
 - Fill rate
 - Revenue
 - Plant performance
 - Product analysis
-

Tab 2: Analytics

Provides:

- Revenue trends
 - Warehouse utilization
 - Risk analysis
 - Shipping cost analysis
 - Carbon emission estimation
 - Sensitivity analysis
 - Benchmark comparison
-

Tab 3: Orders

Provides:

- Optimized order assignments
 - Recommendation explanation
 - Downloadable results
-

Tab 4: About Project

Provides:

- Mathematical formulation
 - Optimization workflow
 - Business interpretation
-

Tab 5: Quantum Optimization

Provides:

- QAOA results
- Quantum assignment results

- Classical vs Quantum comparison

Tab 6: Qora AI Assistant

Provides:

- DOM explanations
- QAOA explanations
- Optimization knowledge
- Chat history
- Conversation export
- Decision support

15. Results and Evaluation

Performance Comparison

Metric	Classical	Quantum
Fill Rate	Lower	Higher
Revenue	Lower	Higher
Runtime	Higher	Optimized
Penalty Cost	Higher	Lower

16. Business Impact

The system helps supply chain planners:

- ✓ Make faster fulfillment decisions
- ✓ Reduce operational expenses
- ✓ Improve customer satisfaction
- ✓ Balance warehouse utilization

- ✓ Reduce penalty costs
 - ✓ Understand optimization decisions
-

17. Screenshots

(Add application screenshots here)

Dashboard

Insert Dashboard Screenshot

Analytics

Insert Analytics Screenshot

Quantum Optimization

Insert Quantum Screenshot

Qora AI

Insert AI Assistant Screenshot

18. Limitations

Current limitations:

1. Dataset is simulated/historical and not connected to live enterprise systems.
 2. Quantum execution currently uses simulator-based execution.
 3. Large-scale quantum hardware testing requires future implementation.
 4. Real-time demand forecasting is not included.
-

19. Future Improvements

Future enhancements:

1. Real-Time Enterprise Integration

Integration with:

- SAP
- ERP Systems
- Supply chain databases

2. AI Demand Forecasting

Use machine learning models to predict:

- Future demand
- Inventory requirements

3. Real Quantum Hardware

Execute QAOA on:

- IBM Quantum processors

4. Advanced Optimization

Include:

- Multi-product optimization
 - Multi-country logistics
 - Real-time decision making
-

20. Installation Guide

Clone Repository

```
git clone <repository-url>
```

Navigate Project

```
cd distributed-order-management-Ai-platform
```

Install Dependencies

```
pip install -r requirements.txt
```

Run Application

```
streamlit run app.py
```

21. Conclusion

This project demonstrates the integration of Artificial Intelligence, Classical Optimization, and Quantum Computing techniques for solving Distributed Order Management challenges.

The developed Hybrid Classical + Quantum Optimization platform provides:

- Intelligent order allocation
- Business-aware decisions
- Interactive analytics
- Quantum optimization exploration
- Explainable recommendations through Qora AI

The solution demonstrates how emerging quantum technologies can support future intelligent supply chain systems.
